

Speaker Notes

NLP Fundamentals · Batch 6

Naskah lisan per slide + panduan menyebut istilah

Navasena AI

8 September 2026

Catatan pengajar untuk mendampingi deck `module03_slides.pdf` (70 slide, hari 3). Tiap entri diberi label **Slide N/70** sesuai nomor halaman deck, berisi naskah yang bisa langsung diucapkan (**Ucapkan**) dan, kalau ada batas yang mudah salah dipahami, penegasannya (**Batas**). Naskah ini bukan bacaan ulang teks slide: slide memuat yang perlu terlihat, notes menyambungkan idenya dan menyebut sampai mana klaimnya berlaku. Frame bertanda *Intuisi* dibawa dengan tempo santai; frame mekanisme (tabel, diagram berlapis) dibaca lebih pelan dengan jeda setelah tiap butir. Sapaan yang dipakai: “kamu” saat mengajak mencoba, “kita” saat menalar bersama. Modul ini punya satu benang merah yang perlu terus disebut: peserta bergerak dari *menghitung kata* menuju *mengukur makna*. Act 2 sampai 4 masih menghitung; Act 5 baru mengganti hitungan dengan vektor makna, dan di situlah jembatan ke Modul 04 dan 05 terbuka. Kalau waktu menipis, yang boleh dipercepat adalah Act 3; yang tidak boleh dilewati adalah Slide 35 (kebocoran data) dan Act 5.

Glosarium: cara menyebut istilah

Pakai pembacaan ini konsisten sepanjang hari, termasuk saat menjawab pertanyaan.

Istilah / singkatan	Dibaca (lantang)
NLP	dieja: “N L P” (<i>natural language processing</i>)
POS tagging	“P O S tagging”, jelaskan sekali sebagai penandaan kelas kata
NER	dieja: “N E R” (<i>named entity recognition</i>)
TF-IDF	dieja: “T F I D F”; jangan dibaca “tif-idf”
BPE	dieja: “B P E” (<i>byte pair encoding</i>)
nlp-id	“N L P dash I D”, nama library-nya
IndoBERT	“Indo-bert”
RAPIDS, cuDF, cuML	“rapids”, “cu-D-F”, “cu-M-L”
nvtex	“N V text”
MinHash	“min-hash”

Istilah / singkatan	Dibaca (lantang)
GPU, CPU, API	dieja: “G P U”, “C P U”, “A P I”
LLM, RAG	dieja: “L L M”, “R A G”
<i>cosine similarity</i>	“cosine similarity”; jelaskan sebagai kemiripan arah
<i>embedding</i>	“embedding”; padanan lisan: “vektor makna”
<i>fine-tune</i>	“fine-tune”; padanan lisan: “dilatih lanjut”
<i>baseline</i>	“baseline”; padanan lisan: “angka pembandingan”
±	“kurang lebih”

Naskah per slide

Slide 1/70 Judul: NLP Fundamentals

Ucapkan: “Hari ini kita masuk ke bahasa. Dua hari kemarin datanya berupa angka dan gambar, dan keduanya sudah berbentuk tabel atau piksel sejak awal. Teks tidak begitu. Panjang kalimatnya berbeda-beda, satu kata bisa punya banyak arti, dan tidak ada satu pun kolom yang siap dimasukkan ke model. Seluruh hari ini pada dasarnya menjawab satu pertanyaan: bagaimana mengubah teks menjadi angka tanpa kehilangan maknanya.”

Slide 2/70 Act 1: Apa itu NLP?

Ucapkan: “Act pertama singkat saja: apa yang dikerjakan NLP, di mana ia dipakai, dan kenapa bahasa Indonesia punya kesulitan yang tidak dimiliki bahasa Inggris.”

Slide 3/70 Definisi NLP

Ucapkan: “NLP adalah cabang AI yang membuat komputer memahami dan memproses bahasa manusia. Kata kuncinya *memahami* — karena kalau cuma memotong dan menyambung string, itu sudah bisa dilakukan sejak dulu tanpa AI. Yang membuatnya NLP adalah usaha menangkap makna: bahwa ‘restoran ini lambat’ itu keluhan, dan ‘antrean panjang tapi worth it’ itu pujian. Contoh yang setiap hari kita pakai: mesin pencari, Google Translate, Siri, dan ChatGPT. Semuanya NLP.”

Slide 4/70 Aplikasi NLP di dunia nyata

Ucapkan: “Enam kategori ini menutupi hampir semua produk NLP yang akan kamu temui. Yang menarik: pola dasarnya cuma satu — teks bebas masuk, sesuatu yang bisa ditindaklanjuti keluar. Kadang keluarannya label (spam atau bukan), kadang angka (skor sentimen), kadang teks lain (terjemahan, ringkasan). Hari ini kita mengerjakan tiga dari enam: analisis sentimen, klasifikasi

teks, dan pencarian.”

Slide 5/70 Tantangan bahasa manusia

Ucapkan: ”Di sinilah bahasa Indonesia berbeda. Bahasa Inggris morfologinya sederhana: `run`, `runs`, `running` — tiga bentuk, imbuhan hanya tinggal ditempel di belakang. Bahasa Indonesia membangun kata dari depan, belakang, dan kadang keduanya sekaligus: `jalan`, `menjalani`, `dijalankan`, `perjalanan`. Konsekuensinya praktis: tool yang dilatih untuk bahasa Inggris tidak bisa dipindahkan begitu saja. Itu sebabnya hari ini kita pakai dua kelompok library — `nlk` dan `spaCy` untuk contoh berbahasa Inggris, `nlp-id` untuk Indonesia.”

Slide 6/70 Intuisi: satu kata dasar, belasan bentuk

Ucapkan: ”Lihat gambarnya sebentar. Tujuh kata di sekeliling, satu kata dasar di tengah: `ajar`. Bagi kita ini jelas satu gagasan. Bagi mesin yang menghitung kata, ini tujuh kolom berbeda yang tidak saling berhubungan — dan tiap kolom harus dipelajari dari nol dengan datanya sendiri. Itulah masalah yang mau diselesaikan lemmatisasi nanti di Slide 13. Simpan gambar ini di kepala; kita akan kembali ke sini dua kali hari ini, sekali di lemmatisasi dan sekali lagi di tokenisasi subword.”

Slide 7/70 Peta perjalanan modul

Ucapkan: ”Empat bagian, dan urutannya sengaja. A membersihkan teks sampai jadi token yang layak. B menggambarkan isinya: kata apa yang dominan, siapa yang disebut. C mulai memberi penilaian: nada kalimat ini apa, dokumen ini masuk kelas mana. D adalah lompatannya — di situ kita berhenti menghitung kata dan mulai mengukur makna. Kalau hari ini kamu cuma bisa mengingat satu bagian, ingat D, karena Modul 04 dan 05 berdiri di atasnya. Notebook 01 menempuh keempatnya di CPU; notebook 02 mengulang alur yang sama di GPU.”

Slide 8/70 Act 2: Membersihkan teks

Ucapkan: ”Act 2 adalah bagian yang paling sering dianggap pekerjaan rumah tangga, dan paling sering jadi penyebab hasil analisis tidak berarti apa-apa. Saya akan tunjukkan buktinya di Act 3.”

Slide 9/70 Pipeline NLP: dari teks ke model

Ucapkan: ”Ini peta jalannya. Teks mentah di kiri, model di kanan, dan di antaranya empat sampai enam langkah yang urutannya cukup baku. Yang perlu kamu ingat: setiap langkah membuang sesuatu. Huruf besar dibuang, tanda baca dibuang, stopword dibuang. Setiap pembuangan itu keputusan, dan setiap keputusan punya kasus di mana ia salah. Kita akan bahas dua kasus seperti itu di slide-slide berikutnya.”

Slide 10/70 Tokenisasi: memecah teks

Ucapkan: "Token adalah unit terkecil yang diproses model. Hari ini kata 'token' akan berubah artinya satu kali, jadi saya sebutkan sekarang supaya tidak membingungkan nanti: di Bagian A, satu token sama dengan satu kata; di Bagian D, satu token adalah potongan subword yang lebih kecil dari kata. Keduanya benar, hanya konteksnya berbeda. Mulai Modul 04 besok, yang dipakai selalu arti yang kedua."

Slide 11/70 Intuisi: kenapa huruf kecil dan tanda baca dibuang duluan

Ucapkan: "Kiri: tanpa normalisasi, NLP, nlp, NLP berkoma, dan NLP dalam kutip dihitung sebagai empat kata berbeda. Kanan: setelah normalisasi, satu kata dengan frekuensi empat. Kenapa ini penting? Karena tiap variasi ejaan yang tidak disatukan menjadi satu dimensi fitur tambahan, dan tiap dimensi tambahan harus dipelajari model dari datanya sendiri. Dengan data terbatas, memecah satu kata jadi empat berarti membagi bukti yang kita punya jadi empat."

Batas: untuk analisis media sosial, huruf kapital semua kadang justru sinyal emosi — di kasus itu jangan buang informasinya begitu saja, catat dulu sebagai fitur terpisah.

Slide 12/70 Stopword: kata yang tidak membedakan

Ucapkan: "Stopword adalah kata yang muncul di hampir semua dokumen: yang, di, dari, dan. Logikanya sederhana — kalau sebuah kata ada di semua dokumen, kehadirannya tidak memberi tahu apa pun tentang dokumen mana yang sedang kita lihat. Daftar stopwords nlp-id sudah disusun untuk bahasa Indonesia, jadi kamu tidak perlu membuatnya sendiri."

Batas: baca kotak Jebakan di slide dengan lantang. Kata **tidak** ada di banyak daftar stopwords. Untuk analisis sentimen, membuangnya berarti 'tidak enak' dan 'enak' menjadi identik. Stopword bukan daftar universal; ia bergantung pada tugasnya.

Slide 13/70 Lemmatisasi vs stemming

Ucapkan: "Dua cara mengembalikan kata ke bentuk dasar. Stemming memotong imbuhan secara mekanis — cepat, tanpa kamus, tapi hasilnya boleh jadi bukan kata sama sekali. Lemmatisasi memakai kamus dan aturan morfologi, jadi keluarannya selalu kata yang benar-benar ada. Untuk bahasa Indonesia yang imbuhanannya berlapis, selisih kualitasnya terasa. Ini kembali ke gambar Slide 6: lemmatisasi adalah usaha menyatukan tujuh cabang tadi kembali ke akarnya."

Batas: sebutkan soal Sastrawi secara terus terang. Nama itu masih sering muncul di tutorial Indonesia, tapi rilis terakhirnya tahun 2016. Boleh dipakai kalau sudah jalan di proyek lama; jangan dipilih untuk proyek baru.

Slide 14/70 POS tagging

Ucapkan: "POS tagging menandai kelas kata: kata benda, kerja, sifat, preposisi. Gunanya bukan untuk dipamerkan, tapi untuk menyaring. Kalau kamu mau menemukan topik sebuah dokumen, ambil kata bendanya saja — kata kerja dan kata sambung jarang menjadi topik. Kalau kamu mau menganalisis tindakan dalam laporan insiden, ambil kata kerjanya."

Batas: label di slide memakai tagset UD supaya mudah dibaca. `nlp-id` mengeluarkan tagset berbeda: di run notebook yang muncul PR, ADV, VB, NN, IN, dan SYM. Sebutkan sekarang supaya tidak bingung. Tambahkan juga bahwa pada kalimat contoh, `alami` ditandai VB padahal ia kata sifat. POS tagger itu model statistik yang menebak, dan tebakannya bisa meleset.

Slide 15/70 Tokenisasi frasa

Ucapkan: "Masalahnya begini: 'Universitas Indonesia' dipecah jadi 'universitas' dan 'indonesia', dan seketika nama lembaganya hilang. Yang tersisa dua kata umum yang bisa muncul di mana saja. `PhraseTokenizer` mengelompokkan frasa nomina jadi satu token, sehingga entitas multi-kata tetap utuh. Berguna sekali sebelum analisis frekuensi."

Batas: ini bukan NER. Ia menemukan bahwa dua kata itu satu frasa, tapi tidak memberi tahu bahwa itu nama organisasi. Untuk labelnya, tunggu Slide 21.

Slide 16/70 Ringkasan Act 2

Ucapkan: "Lima langkah, satu keluaran: daftar token bersih. Itu bahan baku semua yang kita kerjakan sesudah ini. Kalau langkah ini asal-asalan, semua angka di act berikutnya ikut tidak berarti — dan itu yang akan saya buktikan di slide berikutnya."

Slide 17/70 Act 3: Menganalisis teks

Ucapkan: "Act 3 masih menggambarkan teks, belum menilai. Dua pertanyaan: apa yang dibicarakan dokumen ini, dan siapa saja yang disebut di dalamnya."

Slide 18/70 Frekuensi kata

Ucapkan: "Menghitung kata yang paling sering muncul adalah cara termurah menebak topik sebuah koleksi dokumen. Gambarnya membandingkan stopwords Inggris dan Indonesia supaya terlihat bahwa kelompok kata yang harus dibuang memang berbeda per bahasa."

Slide 19/70 Intuisi: frekuensi tanpa pembersihan

Ucapkan: "Inilah bukti yang saya janjikan di Act 2. Kiri: lima kata teratas dari teks mentah — yang, di, dan, adalah, untuk. Ambil dokumen apa pun berbahasa Indonesia, hasilnya akan

mirip. Daftar itu tidak memberi tahu apa pun. Kanan: setelah dibersihkan, `nlp`, `bahasa`, `model`, `analisis`, `sentimen` — baru di sini topiknya terbaca. Perbedaannya bukan kosmetik. Yang kiri nol informasi, yang kanan adalah jawabannya.”

Slide 20/70 Word cloud: cantik, tapi hati-hati

Ucapkan: ”Word cloud enak dipandang dan bagus untuk laporan ke pihak yang bukan teknis. Tapi mata manusia buruk sekali dalam membandingkan luas huruf, dan tata letaknya acak — posisi kata di gambar tidak membawa informasi apa pun. Aturan praktisnya di kotak kanan: word cloud untuk menarik perhatian, bar chart untuk mengambil keputusan. Di notebook kita membuat keduanya supaya bedanya terasa.”

Slide 21/70 Named Entity Recognition

Ucapkan: ”NER menemukan dan melabeli entitas yang disebut dalam teks: orang, organisasi, lokasi, tanggal. Bedanya dengan tokenisasi frasa di Slide 15 — di sana kita cuma tahu ’ini satu frasa’, di sini kita tahu ’ini nama organisasi’. Labelnya itu yang membuatnya berguna.”

Batas: sebutkan dua hal. Pertama, `spaCy` melabeli kota dan negara sebagai GPE, bukan LOC, jadi GPE di keluaran notebook bukan kesalahan. Kedua, di run notebook `Nvidia` justru ikut dilabeli GPE padahal itu perusahaan: model kecil `en_core_web_sm` memang sering keliru pada nama perusahaan yang mirip nama tempat. Pakai itu sebagai bahan, keluaran NER adalah tebakan model dan di produksi ia perlu diperiksa.

Slide 22/70 NER untuk apa? Tiga kasus nyata

Ucapkan: ”Tiga kasus ini nyata dan sering diminta klien. Kontrak: tarik nama pihak, nilai, dan tanggal dari ratusan PDF — pekerjaan yang biasanya dilakukan manual sehari-hari. Berita: pantau setiap penyebutan nama perusahaan, lalu gabungkan dengan skor sentimennya, dan kamu punya dashboard reputasi. Layanan: isi otomatis formulir tiket dari keluhan berbentuk teks bebas. Polanya satu: teks bebas masuk, data terstruktur keluar. Itulah nilai jual NER.”

Slide 23/70 NER bahasa Indonesia: tiga jalur

Ucapkan: ”Pertanyaan yang pasti muncul: kalau `spaCy` hanya punya model Inggris, bagaimana dengan Indonesia. Tiga jalur di tabel. Model multibahasa `spaCy` ringan tapi akurasinya sedang. Model NER berbasis `IndoBERT` di Hugging Face akurasinya paling baik untuk Indonesia, tapi unduhannya besar dan butuh GPU agar cepat. `PhraseTokenizer` adalah jalan pintas tanpa model — sangat cepat, tapi tidak memberi label.”

Batas: tegaskan bahwa `spaCy` memang belum menyediakan pipeline khusus bahasa Indonesia. Ini bukan kekurangan materi, melainkan keadaan ekosistemnya.

Slide 24/70 Ringkasan Act 3

Ucapkan: "Empat butir, dan yang terakhir jadi jembatan: sampai di sini kita baru menggambarkan teks. Act 4 mulai menilainya — dan di situ ada satu kesalahan yang sering lolos dari review."

Slide 25/70 Act 4: Menilai teks

Ucapkan: "Act 4 punya dua bagian: analisis sentimen, lalu klasifikasi teks. Di ujungnya ada satu slide yang menurut saya paling penting sepanjang hari ini, dan itu bukan tentang NLP — melainkan tentang cara mengevaluasi model apa pun."

Slide 26/70 Analisis sentimen: dua generasi

Ucapkan: "Dua pendekatan, dan membandingkannya langsung adalah cara tercepat memahami kenapa NLP pindah ke transformer. Generasi pertama, leksikon: simpan kamus kata beserta skornya, lalu jumlahkan skor kata yang muncul. Cepat, transparan, jalan di laptop mana pun. Generasi kedua, transformer: model dilatih dari teks nyata berlabel, dan ia membaca susunan kalimat, bukan menjumlah. Harganya unduhan ratusan MB dan komputasi yang jauh lebih berat."

Slide 27/70 Skala polaritas

Ucapkan: "TextBlob mengeluarkan dua angka. Polarity dari minus satu sampai plus satu: negatif ke positif. Subjectivity dari nol sampai satu: faktual ke opini. Yang kedua sering dilupakan padahal berguna — kalimat 'harganya Rp50.000' punya subjectivity rendah, dan kalau kamu memfilter dengan itu, kamu bisa memisahkan pernyataan fakta dari pendapat."

Batas: baca kotak Jebakan pelan-pelan. Skor nol punya dua arti yang sangat berbeda — kalimatnya memang netral, atau tidak ada satu pun katanya yang ada di kamus. Leksikon tidak membedakan keduanya, dan tidak memberi peringatan. Slide 29 menunjukkan akibatnya.

Slide 28/70 Batas leksikon: negasi

Ucapkan: "Baris pertama justru benar, dan itu penting disebut supaya kita jujur: 'The plot was not good' diberi minus nol koma tiga lima. TextBlob memang punya aturan negasi — **not** membalik skor kata tepat sesudahnya. Batasnya baru terlihat di baris kedua. 'I would not call this a great movie' itu kritik, tapi skornya plus nol koma delapan, positif kuat. Sebabnya **not** berjarak tiga kata dari **great**, di luar jangkauan aturannya, jadi yang terhitung tinggal **great**. Baris ketiga kebalikannya: 'Nobody would say this restaurant is bad' itu pujian, tapi diberi minus nol koma tujuh."

Batas: jangan menyederhanakannya jadi 'leksikon tidak bisa menangani negasi'. Ia bisa, untuk negasi yang berdekatan. Yang tidak bisa ditangani adalah negasi berjarak dan sarkasme, dan ketiga angka di tabel ini diukur langsung, bukan dikarang.

Slide 29/70 Batas leksikon: bahasa

Ucapkan: "Sekarang masalah yang lebih dekat ke kita. Kalimat 'Pelayanan restoran itu lambat dan mengecewakan' jelas negatif bagi kita. TextBlob mengembalikan nol koma nol nol."

Batas: jeda sebentar di sini, lalu baca kotak Jebakan. Nol itu bukan 'netral'. Kamus TextBlob hanya berbahasa Inggris, jadi tidak ada satu pun kata Indonesia yang dikenali. Nol berarti 'tidak tahu' — dan model tidak akan pernah memberi tahu kamu bedanya. Ini kegagalan diam, jenis yang paling berbahaya, karena pipeline-nya jalan mulus dan angkanya tetap keluar.

Slide 30/70 Sentimen Indonesia dengan IndoBERT

Ucapkan: "Beginilah cara industri Indonesia mengerjakannya sekarang. Empat baris kode: panggil `pipeline`, sebut nama modelnya, masukkan kalimat. Modelnya sudah dilatih lanjut pada data sentimen berbahasa Indonesia oleh orang lain, dan kita tinggal memakainya. Pola ini persis transfer learning yang kemarin kamu lihat dengan ResNet di Modul 02 — model besar yang sudah dilatih, dipakai ulang untuk tugas kita."

Batas: sebutkan bahwa model ini mengenali tiga kelas, termasuk netral. Banyak model sentimen hanya punya dua, dan itu memaksa kalimat faktual masuk ke salah satunya.

Slide 31/70 Intuisi: kenapa transformer tahan bahasa gaul

Ucapkan: "Di notebook, kalimat 'Gw suka bgt sama teknologi NLP nih' langsung dikenali positif — kalimat yang gagal total dinormalkan pipeline klasik kita di Section 1. Kenapa? Bukan karena modelnya lebih pintar, tapi karena pengetahuannya datang dari tempat lain. Kamus leksikon disusun manusia dari teks baku, dan `gw` tidak ada di sana. Transformer dilatih dari teks nyata — Twitter, ulasan, forum — jadi bahasa gaul ikut terlatih karena memang ada di dalamnya. Ditambah tokenisasi subword, kata yang belum pernah dilihat pun tetap terwakili sebagian."

Slide 32/70 Leksikon atau transformer?

Ucapkan: "Jangan buru-buru menyimpulkan transformer selalu menang. Kolom kiri: kalau datanya jutaan dokumen dan kamu cuma butuh penyaringan kasar, kalau tidak ada GPU, kalau skornya harus bisa dijelaskan per kata untuk audit — leksikon masih pilihan yang benar. Kolom kanan: teks Indonesia, bahasa informal, negasi penting, butuh kelas netral yang jujur. Dan yang paling sering dipakai di produksi: keduanya berlapis — leksikon menyaring dulu supaya murah, transformer memutuskan yang tersisa."

Slide 33/70 Klasifikasi teks: bag-of-words

Ucapkan: "Sekarang kita berhenti memakai model orang lain dan melatih classifier sendiri. Langkah pertama: mengubah dokumen jadi angka. Bag-of-words caranya paling sederhana — untuk tiap kata di daftar fitur, catat ada atau tidak ada. Urutan kata dibuang total. Namanya

'kantong kata' memang karena itu: seolah semua kata dimasukkan ke kantong lalu dikocok. Sederhana, tapi ini bentuk paling awal dari gagasan besar hari ini — teks sebagai vektor angka."

Slide 34/70 Naive Bayes: kenapa masih diajarkan

Ucapkan: "Naive Bayes menghitung peluang tiap kelas lewat Teorema Bayes. Disebut 'naive' karena ia menganggap kemunculan tiap kata saling bebas — asumsi yang jelas salah, karena 'nasi' dan 'goreng' sangat berhubungan. Anehnya ia tetap bekerja baik untuk teks. Alasan ia masih diajarkan bukan akurasi, melainkan kolom kanan itu: keputusannya bisa dibaca manusia. **outstanding** muncul hampir dua belas kali lebih sering di ulasan positif — itu kalimat yang bisa kamu bawa ke rapat. Neural network tidak memberimu itu."

Slide 35/70 Intuisi: kebocoran data

Ucapkan: "Ini slide terpenting hari ini, dan kalau waktu mepet, potong slide lain, jangan yang ini. Fitur kita adalah dua ribu kata paling sering muncul. Pertanyaannya: sering muncul di data yang mana? Kolom merah, cara yang salah — hitung dari seluruh korpus dulu, baru pecah train dan test. Kelihatan wajar, dan kodenya jalan tanpa error. Tapi data test sudah ikut menentukan kolom mana yang dipakai model, jadi model sudah 'mengintip' sebelum diuji, dan akurasi jadi terlalu bagus. Kolom hijau, cara yang benar: pecah dulu, pilih fitur dari train saja, baru terapkan ke test."

Batas: tekankan bahwa ini bukan kesalahan langka. Notebook modul ini versi lama melakukan persis kesalahan yang di kolom merah, dan tidak ada yang menyadarinya sampai diaudit. Kebocoran data tidak memunculkan error — ia cuma memberi angka yang menyenangkan.

Slide 36/70 Aturan yang sama berlaku untuk semuanya

Ucapkan: "Kebocoran tadi bukan cuma soal seleksi fitur. Tabel ini daftar langkah yang sama-sama 'belajar' sesuatu dari data. `TfidfVectorizer` belajar kosakata dan bobot IDF. `StandardScaler` belajar rata-rata dan simpangan baku. Imputasi nilai kosong belajar median atau modus. Semuanya di-fit pada train, lalu diterapkan ke test — tanpa kecuali. Aturan ini akan terus berlaku di Modul 04 dan 05, jadi hafalkan bentuknya, bukan contohnya."

Slide 37/70 Baseline: angka pembanding yang wajib

Ucapkan: "Akurasi 78 persen itu bagus atau jelek? Tidak ada yang bisa menjawab tanpa angka pembandingnya. Baseline kelas mayoritas adalah akurasi yang kamu dapat kalau selalu menebak kelas terbanyak, tanpa model sama sekali. Korpus `movie_reviews` seimbang seribu lawan seribu, dan pada pembagian test di notebook baselinenya jatuh di 54,8 persen. Selisih 23 poin itu berarti model benar-benar belajar sesuatu."

Batas: berikan contoh sebaliknya supaya menempel. Kalau 95 persen datamu satu kelas, model

dengan akurasi 94 persen sebenarnya lebih buruk daripada menebak asal. Angka 94 terdengar hebat sampai kamu tahu pembandingnya.

Slide 38/70 Ringkasan Act 4

Ucapkan: "Lima butir. Yang paling perlu dibawa pulang: dua yang tengah tentang evaluasi, bukan tentang NLP. Dan butir penutupnya membuka Act 5 — masalah yang belum kita selesaikan adalah bahwa semua hitungan kata di atas tidak tahu 'suka' dan 'gemar' itu sama."

Slide 39/70 Act 5: Dari kata ke makna

Ucapkan: "Act 5 adalah puncak hari ini. Kalau kamu memahami satu act saja, pilih yang ini — karena Modul 04 dan Modul 05 seluruhnya berdiri di atas gagasan yang dibangun di sini."

Slide 40/70 Tiga anak tangga representasi teks

Ucapkan: "Tiga anak tangga, dan ketiganya mengerjakan hal yang sama: mengubah teks jadi angka. Yang membedakan adalah berapa banyak makna yang ikut terbawa. Bag-of-words cuma tahu ada atau tidak ada. TF-IDF menambahkan bobot, jadi kata langka dihargai lebih tinggi. Embedding menambahkan pengetahuan dari jutaan kalimat, dan di situlah makna mulai masuk."

Slide 41/70 TF-IDF: kata langka lebih berharga

Ucapkan: "TF adalah seberapa sering kata muncul di dokumen ini. IDF adalah seberapa jarang kata itu di seluruh koleksi. Kalikan keduanya, dan kata yang sering di sini tapi jarang di tempat lain dapat bobot tinggi. Lihat contoh di kanan: **yang** ada di seratus persen dokumen, bobotnya mendekati nol. **minhash** ada di dua persen, bobotnya tinggi. Menariknya, IDF melakukan otomatis apa yang tadi kita lakukan manual dengan daftar stopword — menekan kata yang ada di mana-mana. Bedanya ia menyesuaikan diri dengan koleksi dokumenmu sendiri."

Slide 42/70 TF-IDF buta terhadap sinonim

Ucapkan: "Dua kalimat: 'Saya suka makan nasi goreng' dan 'Aku gemar menyantap nasi goreng'. Maknanya hampir identik. Skor TF-IDF-nya nol koma dua tiga. Kenapa? Karena bagi TF-IDF, **suka** dan **gemar** adalah dua kolom berbeda yang tidak berhubungan sama sekali — sama tidak berhubungannya dengan **suka** dan **beton**. Yang tersisa untuk dicocokkan cuma **nasi** dan **goreng**. Embedding memberi nol koma sembilan enam untuk pasangan yang sama."

Batas: angka ini diukur langsung di notebook Section 9 dengan model `paraphrase-multilingual-MiniLM-L12-v2`. Model embedding lain akan memberi angka berbeda; yang tetap adalah arah selisihnya.

Slide 43/70 Intuisi: embedding sebagai koordinat

Ucapkan: "Beginilah cara membayangkannya. Model menempatkan tiap kata atau kalimat di sebuah ruang. Kata bermakna mirip diletakkan berdekatan: *suka*, *gemar*, *menyantap* berkumpul di satu sudut; *harga* dan *BBM* di sudut lain. Begitu makna jadi posisi, kemiripan makna jadi jarak — dan jarak bisa dihitung. Itu lompatan besarnya: sesuatu yang tadinya harus ditebak manusia sekarang bisa dihitung mesin."

Batas: gambar ini dua dimensi supaya bisa dilihat. Ruang sebenarnya 384 dimensi, dan tidak ada yang bisa membayangkannya. Yang berlaku sama di kedua ruang cuma satu: yang dekat itu mirip.

Slide 44/70 Satu kalimat, 384 angka

Ucapkan: "Masukkan kalimat, keluar 384 angka. Yang perlu diperhatikan: panjang kalimatnya berapa pun, keluarannya selalu 384. Kalimat lima kata dan kalimat lima puluh kata sama-sama jadi 384 angka. Itu yang membuat kalimat bisa dibandingkan satu sama lain — panjangnya seragam. Model yang kita pakai multibahasa, jadi kalimat Indonesia dan terjemahan Inggrisnya diletakkan berdekatan. Itu berguna sekali: kamu bisa mencari dokumen Inggris dengan query Indonesia."

Slide 45/70 Cosine similarity

Ucapkan: "Kalau makna sudah jadi arah, kemiripan jadi sudut. Cosine similarity mengukur sudut antar vektor, bukan panjangnya. Nilai satu berarti arahnya sama persis; nol berarti tegak lurus, tidak berhubungan. Kenapa sudut dan bukan jarak biasa? Karena panjang vektor sering mencerminkan panjang dokumen, dan kita tidak mau dokumen panjang otomatis dianggap berbeda dari yang pendek. Dengan sudut, keduanya adil dibandingkan."

Slide 46/70 Semantic search dalam sepuluh baris

Ucapkan: "Empat langkah, dan itu seluruh mesinnya. Encode dokumen, encode query, hitung cosine similarity, ambil yang skornya tertinggi. Di notebook, query 'puncak paling tinggi di Indonesia' menemukan dokumen 'Gunung Kerinci adalah gunung berapi tertinggi di Indonesia' — dan tidak ada satu pun kata yang sama antara keduanya. Tidak ada 'gunung', tidak ada 'Kerinci', tidak ada 'tertinggi' yang persis. TF-IDF tidak akan pernah menemukan dokumen itu."

Slide 47/70 Inilah inti RAG

Ucapkan: "Kiri: yang barusan kamu bangun hari ini. Kanan: RAG di Modul 05. Bandingkan baris per baris — hampir sama persis. Yang bertambah cuma dua: dokumen disimpan di FAISS supaya pencarian tetap cepat untuk jutaan dokumen, dan hasil pencariannya diserahkan ke LLM untuk disusun jadi kalimat jawaban. Jadi kalau minggu depan ada yang bilang RAG itu rumit, ingat slide ini: bagian intinya sudah kamu tulis hari ini dalam sepuluh baris."

Slide 48/70 Tokenisasi subword

Ucapkan: "Di sinilah arti kata 'token' berubah, seperti yang saya sebut di Slide 10. LLM tidak memecah teks per kata, melainkan per potongan subword — lebih kecil dari kata, lebih besar dari huruf. `mempelajari` jadi empat potongan. Kenapa repot-repot begitu? Karena dengan kosakata terbatas, tiga puluh sampai lima puluh ribu subword, model bisa menuliskan kata apa pun, termasuk kata yang belum pernah dilihatnya saat training. Nama orang, istilah baru, typo — semuanya tetap bisa direpresentasikan dari potongan yang sudah dikenal."

Batas: sambungkan lagi ke Slide 6. Untuk bahasa Indonesia yang kaya imbuhan, subword punya bonus: `mempelajari` dan `dipelajari` berbagi potongan yang sama, jadi model tidak perlu belajar keduanya dari nol.

Slide 49/70 Intuisi: token adalah satuan biaya

Ucapkan: "Ini bagian yang langsung terasa di dompet. Dua pasang kalimat yang maknanya sama; sisi Inggris jadi 16 token, sisi Indonesia jadi 40 token, rasionya dua setengah kali. Penyebabnya bukan bahasa Indonesia lebih rumit, melainkan tokenizer GPT-2 dilatih dominan pada teks Inggris, jadi kata Indonesia terpaksa dipecah lebih halus. Akibatnya: API LLM menagih per token, sehingga isi yang sama jadi dua setengah kali lebih mahal. Tokenizer multibahasa menekan rasio itu jadi 1,35 kali pada kalimat yang sama, dan itu pertimbangan nyata saat memilih model."

Batas: rasio ini bergantung pada kalimat yang dipakai. Minta peserta menghitung ulang dengan kalimat mereka sendiri di notebook Section 10; angkanya akan berbeda, arahnya tidak.

Slide 50/70 "Token" mulai Modul 04

Ucapkan: "Tabel penutup untuk menghilangkan kebingungan. Di Bagian A hari ini, satu token sama dengan satu kata. Di Bagian D dan seterusnya, satu token sama dengan satu subword. Mulai besok di Modul 04, setiap kali kamu melihat `max_new_tokens`, batas konteks, atau tagihan API — yang dimaksud selalu subword."

Slide 51/70 Ringkasan Act 5

Ucapkan: "Lima butir, dan semuanya berjalan di CPU tanpa masalah karena datanya kecil. Act 6 mengambil alur yang sama dan menjalankannya pada skala yang membuat CPU menyerah."

Slide 52/70 Act 6: NLP di GPU

Ucapkan: "Act 6 mendampingi notebook kedua. Pastikan peserta sudah mengganti runtime ke T4 GPU sebelum act ini dimulai — pergantian runtime menghapus variabel, jadi lebih baik dilakukan sekarang daripada di tengah."

Slide 53/70 Kenapa NLP klasik butuh GPU

Ucapkan: "Pertanyaan yang wajar: bukankah GPU itu untuk deep learning? Untuk NLP klasik pun ia berguna, dan alasannya ada di gambar kanan. CPU punya belasan core yang masing-masing sangat cepat mengolah satu string. GPU punya ribuan core yang lebih lambat, tapi bisa mengolah ribuan string sekaligus. Operasi teks — tokenisasi, hitung n-gram, normalisasi — persis pekerjaan seragam berulang yang cocok untuk pola kedua. Dan menyiapkan korpus training LLM berarti memproses miliaran dokumen, jadi perbedaannya bukan puluhan detik, melainkan berjam-jam lawan menit."

Slide 54/70 Stack RAPIDS untuk NLP

Ucapkan: "Tiga library, dan dua di antaranya punya padanan yang sudah kamu kenal. cuDF adalah pandas versi GPU. cuML adalah scikit-learn versi GPU. Yang menarik justru baris tengah: `nvtext` tidak punya padanan di pandas sama sekali, karena operasinya memang lahir di GPU. Kabar baiknya, ketiganya sudah terpasang di runtime GPU Colab — tinggal `import`, tanpa instalasi."

Slide 55/70 Nol perubahan kode

Ucapkan: "Ini bagian yang paling mengejutkan peserta. File Python yang sama, tanpa satu baris pun diubah, dijalankan dua kali. Yang pertama biasa, yang kedua lewat dua akselerator. Yang pertama pakai CPU, yang kedua pakai GPU. Caranya bukan mengubah kode, melainkan mengubah cara menjalankannya — karena akselerator harus aktif sebelum `import pandas` dieksekusi. Operasi yang belum didukung GPU otomatis kembali ke CPU, jadi kodenya tidak pernah crash gara-gara ini."

Slide 56/70 Intuisi: GPU tidak selalu menang

Ucapkan: "Jangan biarkan peserta pulang dengan kesimpulan 'GPU selalu lebih cepat'. Lihat grafiknya. Garis GPU mulai lebih tinggi, karena GPU membayar biaya tetap: inisialisasi CUDA dan pemindahan data bolak-balik. Untuk data kecil, biaya tetap itu lebih mahal daripada komputasinya sendiri, dan di kiri titik impas GPU justru kalah. Konsekuensi praktisnya ada di butir terakhir, dan ini aturan emas: jangan pernah mengukur pemanggilan GPU pertama. Lakukan warm-up dulu."

Batas: sebutkan bahwa notebook memang memuat sel warm-up khusus, dan tanpa sel itu cuML bisa tampak lebih lambat dari scikit-learn. Kalau ada peserta yang melaporkan GPU-nya kalah, pertanyaan pertama: sel warm-up-nya kelewat atau tidak.

Slide 57/70 **nvtext: operasi yang tidak ada di pandas**

Ucapkan: "Empat operasi di tabel, dan tidak satu pun punya padanan langsung di pandas. `token_count` menghitung token per baris tanpa perlu membentuk daftarnya dulu — hemat memori. `ngrams_tokenize` membangun bigram langsung di GPU. `minhash` membuat sidik jari dokumen, dan itu yang kita pakai di babak berikutnya. Polanya: makin intensif operasi string-nya, makin lebar kemenangan GPU."

Slide 58/70 **Seberapa cepat?**

Ucapkan: "Angka ini diukur langsung dari notebook 02 di Colab T4 dengan 150 ribu paragraf Wikipedia Indonesia. Perhatikan bahwa selisihnya berbeda per operasi — bukan satu angka ajaib yang berlaku untuk semuanya."

Batas: katakan terus terang bahwa angka di mesin peserta akan berbeda, tergantung GPU yang sebagian di Colab dan beban runtime saat itu. Yang perlu mereka bawa pulang adalah polanya, bukan angkanya.

Slide 59/70 **MinHash: dari miliaran perbandingan**

Ucapkan: "Korpus mentah selalu penuh duplikat: artikel template, boilerplate, salin-tempel. Untuk training LLM itu masalah, karena duplikat membuat model menghafal dan komputasi terbuang. Tapi cara lugu untuk menemukannya mustahil: 150 ribu dokumen berarti sekitar 11 miliar pasangan yang harus dibandingkan. MinHash mengambil jalan pintas — tiap dokumen diringkas jadi sidik jari kecil, lalu tinggal dikelompokkan yang sidik jarinya sama. Dari miliaran perbandingan jadi satu `groupby`."

Batas: notebook memakai satu sidik jari utuh, jadi hanya duplikat yang nyaris identik tertangkap. Sistem produksi memakai banyak band sidik jari (MinHash-LSH) supaya kemiripan parsial pun terdeteksi. Sebutkan ini supaya peserta tidak mengira versi notebook sudah setara produksi.

Slide 60/70 **Intuisi: kenapa sidik jari cukup**

Ucapkan: "Kenapa mengganti perbandingan dengan sidik jari itu sah? Karena MinHash dirancang supaya dokumen yang mirip menghasilkan sidik jari sama dengan probabilitas tinggi. Begitu itu berlaku, masalah 'cari yang mirip' berubah jadi masalah 'kelompokkan yang sama' — dan yang kedua jauh lebih murah, karena mengelompokkan nilai yang identik adalah operasi paling dasar di database mana pun."

Batas: kata 'probabilitas tinggi' itu penting. MinHash bisa meleset di kedua arah — melewatkan pasangan mirip, atau menyatukan yang sebenarnya berbeda. Untuk penyiapan korpus, kesalahan sesekali masih dapat diterima; untuk audit hukum, tidak.

Slide 61/70 Dari lab ke produksi: NeMo Curator

Ucapkan: "Baris per baris, apa yang peserta kerjakan hari ini dan padanannya di produksi. Tokenisasi dan normalisasi di nvtext menjadi identifikasi bahasa, pembersihan, dan penyaringan kualitas. MinHash plus `groupby` menjadi fuzzy dedup terdistribusi. TF-IDF plus regresi logistik menjadi classifier kualitas dokumen. Yang berbeda cuma skala: satu T4 dan 150 ribu paragraf lawan ratusan GPU dan miliaran dokumen. Langkah-langkahnya sama."

Slide 62/70 Ringkasan Act 6

Ucapkan: "Lima butir, dan dua di antaranya bukan tentang RAPIDS melainkan tentang cara mengukur dengan jujur: GPU menang di skala, dan warm-up wajib. Keduanya berlaku untuk benchmark GPU apa pun, termasuk yang akan kamu lakukan di Modul 06."

Slide 63/70 Act 7: Ringkasan & lanjutan

Ucapkan: "Penutup. Kita rangkum, lalu lihat ke mana modul ini bersambung."

Slide 64/70 Ringkasan Modul 03

Ucapkan: "Sepuluh section notebook 01 plus notebook 02, tersusun per bagian. Pakai tabel ini sebagai daftar periksa: kalau ada baris yang terasa asing, itu bagian yang perlu diulang sendiri sebelum Modul 04 besok."

Slide 65/70 Tiga hal yang dibawa pulang

Ucapkan: "Kalau semua hari ini menguap dan cuma tiga kalimat yang tersisa, biar tiga ini yang bertahan. Satu: pembersihan menentukan segalanya — tanpa membuang stopword, analisis frekuensi apa pun hanya memunculkan *yang* dan *di*. Dua: apa pun yang belajar dari data harus *di-fit* pada train saja, dan melanggarnya tidak memberi error, cuma angka palsu. Tiga: hitungan kata tidak sama dengan makna, dan perbedaan itulah yang membuat RAG mungkin."

Slide 66/70 Notebook 01: peta section

Ucapkan: "Notebook 01 jalan di CPU, sekitar 60 menit termasuk membaca. Empat bagian sesuai peta di Slide 7, ditutup empat soal latihan. Soal nomor dua yang paling saya sarankan dikerjakan — ia meminta kamu mengubah jumlah fitur dari dua ribu jadi tiga ratus dan menjelaskan akibatnya, dan itu memaksa kamu memahami ulang bag-of-words."

Slide 67/70 Notebook 02: peta babak

Ucapkan: "Notebook 02 wajib runtime T4 GPU, sekitar 45 menit. Lima babak, dan datanya dua: SmSA dari IndoNLU untuk sentimen, dan Wikipedia Bahasa Indonesia lewat streaming untuk skala. Streaming berarti tidak ada dump raksasa yang perlu diunduh — cukup ambil sebanyak yang dibutuhkan."

Slide 68/70 Setup & tools

Ucapkan: "Bagian kanan lebih penting daripada kiri, jadi baca yang kanan dulu. Butir pertama adalah jebakan yang nyata: `nlp-id` versi baru mengunci `huggingface-hub` versi 1, dan `transformers` versi 4 tidak kompatibel dengan itu. Kalau kamu memasang `transformers<5`, instalasinya gagal total — bukan sekadar memberi peringatan. Butir kedua: kalau Colab menampilkan tombol restart session, tekan, lalu lanjut dari sel berikutnya. Jangan mengulang `pip install`."

Slide 69/70 Lanjutan: Modul 04 – LLM

Ucapkan: "Tiga sambungan, dan semuanya konkret. Ke Modul 04: `pipeline` yang tadi dipakai untuk IndoBERT adalah pintu masuk yang sama ke LLM, dan kata 'token' yang dipakai di sana adalah subword dari Section 10. Ke Modul 05: pola encode-bandingkan-ambil top-k dari Section 9 dipakai lagi, kali ini dengan FAISS. Ke Modul 06: RAPIDS hari ini bersambung ke NeMo dan TensorRT-LLM."

Slide 70/70 Penutup

Ucapkan: "Terima kasih. Kalau ada satu pertanyaan yang layak dibawa ke sesi berikutnya, itu pertanyaan ini: kalau embedding sudah bisa menangkap makna satu kalimat, apa lagi yang ditambahkan LLM di atasnya? Jawabannya besok."